**A PROJECT REPORT ON**

# SMARTSCHOLAR

SUBMITTED TO
MIT SCHOOL OF COMPUTING, LONI, PUNE IN PARTIAL FULFILLMENT OF
THE REQUIREMENTS FOR THE AWARD OF THE DEGREE

**BACHELOR OF TECHNOLOGY**
**(Computer Science & Engineering)**

**BY**


| | |
|---|---|
| Vedant Barhate | Enrollment No: ADT23SOCB1288 |
| Yash Pandey | Enrollment No: ADT23SOCB1337 |
| Vibhor Dev | Enrollment No: ADT23SOCB1303 |
| Achyut Yesare | Enrollment No: ADT23SOCB0041 |


**Under the guidance of**

Dr. Arvind Jagtap




**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**


**MIT School OF COMPUTING**
**MIT Art, Design and Technology University**
**Rajbaug Campus, Loni-Kalbhor, Pune 412201**


**2024-25**

**MIT SCHOOL OF COMPUTING**
**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**
MIT ART, DESIGN AND TECHNOLOGY UNIVERSITY,
RAJBAUG CAMPUS, LONI-KALBHOR, PUNE 412201

# CERTIFICATE

This is to certify that the project report entitled

**"SMARTSCHOLAR"**

Submitted by

| | |
|---|---|
| Vedant Barhate | Enrollment No: ADT23SOCB1288 |
| Yash Pandey | Enrollment No: ADT23SOCB1337 |
| Vibhor Dev | Enrollment No: ADT23SOCB1303 |
| Achyut Yesare | Enrollment No: ADT23SOCB0041 |

is a bonafide work carried out by them under the supervision of Dr. Arvind Jagtap and it is submitted towards the partial fulfillment of the requirement of MIT ADT university, Pune for the award of the degree of Bachelor of Technology (Computer Science and Engineering)

**Dr. Arvind Jagtap**
Guide

**Dr. Ganesh Pathak**
Head of Department

**Prof. Dr. Vipul Dalal**
Director

**Prof. Dr. Rajneeshkaur Sachdev**
Dean

Seal/Stamp of the College
Place: PUNE

# DECLARATION

We, the team members

| Name | Enrollment No |
|---|---|
| Vedant Barhate | (ADT23SOCB1288) |
| Yash Pandey | (ADT23SOCB1337) |
| Vibhor Dev | (ADT23SOCB1303) |
| Achyut Yesare | (ADT23SOCB0041) |

Hereby declare that the project work incorporated in the present project entitled **"SmartScholar** is original work. This work (in part or in full) has not been submitted to any University for the award or a Degree or a Diploma. We have properly acknowledged the material collected from secondary sources wherever required. We solely own the responsibility for the originality of the entire content.

Date: __/05//2025

Name & Signature of the Team Members

Member 1: _____

Member 2: _____

Member 3: _____

Member 4: _____

**Name and Signature of Guide**

Seal/Stamp of the College

Place: Pune

Date: __/05/2025




DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
MIT SCHOOL OF COMPUTING,
RAJBAUG, LONI KALBHOR,
PUNE – 412201

# EXAMINER'S APPROVAL CERTIFICATE

The project report entitled "SMARTSCHOLAR" submitted by Vedant Barhate (ADT23SOCB1288), Yash Pandey (ADT23SOCB1337), Vibhor Dev (ADT23SOCB1303), Achyut Yesare (ADT23SOCB0041) in partial fulfillment for the award of the degree of Bachelor of Technology (Computer Science & Engineering) during the academic year 2024-25, of MIT-ADT University, MIT School OF COMPUTING, Pune, is hereby approved.

**Examiners:**

**1.**

**2.**

# ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all those who have contributed to the successful development of our project, SmartScholar: An AI-Powered Education Assistant.

First and foremost, we are deeply thankful to our project guide, Dr. Arvind Jagtap, for his invaluable guidance, constructive feedback, and continuous encouragement throughout the development journey. Her mentorship played a crucial role in shaping the project's direction and technical rigor.

We are grateful to the faculty members and staff of MIT School of Computing for providing access to essential resources and fostering an environment conducive to innovation. Their academic input laid a strong foundation for exploring the intersection of AI, natural language processing, and educational support systems.

We extend our heartfelt thanks to our peers and colleagues for their insightful feedback, engaging technical discussions, and constant moral support, all of which contributed significantly to refining the system's design and functionality.

We would also like to acknowledge the open-source developer community and the creators of various libraries and frameworks—such as Python, LangChain, PyMuPDF, and SentenceTransformers—whose tools greatly facilitated efficient development, experimentation, and prototyping.

Lastly, we are immensely thankful to our families and friends for their unwavering support, patience, and motivation throughout this endeavor. Their belief in our vision encouraged us to persist through challenges and deliver SmartScholar with dedication and purpose.

This project stands as a testament to collaborative effort and shared learning, and we are truly appreciative of everyone who played a part in making it a reality.

**Name, Enrollment No.**

Vedant Barhate, ADT23SOCB1288
Yash Pandey, ADT23SOCB1337
Vibhor Dev, ADT23SOCB1303
Achyut Yesare, ADT23SOCB0041

# ABSTRACT

This project presents **Smart Scholar 1.0**, an AI-powered personalized study and research assistant designed to enhance the way students and researchers engage with academic content. In today's educational landscape, learners face increasing volumes of study material—ranging from textbooks and notes to research papers—often without tools that contextualize this information to their individual needs. Smart Scholar addresses this gap by enabling users to link personal study folders and interact with their materials through intelligent, AI-generated assistance.

The system integrates **Retrieval-Augmented Generation (RAG)** with **Natural Language Processing (NLP)** and **Local Language Models (LLMs)** to deliver accurate, context-aware summaries, explanations, and answers. Users can ask questions, receive concept clarifications, generate structured insights, and navigate their study material more efficiently.

Smart Scholar operates in two modes:

- **Offline Mode**, where all processing is done locally using a compact LLM (currently LLaMA 3B via Ollama 3.2), preserving privacy and enabling continuous access.
- **Online Mode**, where the system generates dynamic prompts to interact with external AI services (e.g., ChatGPT, llama 3) for real-time updates and enhanced responses.

Core features include real-time folder monitoring, semantic chunking and embedding, vector-based document retrieval, intent classification, and a session memory system to maintain learning continuity. The current interface is developed using **customtkinter**, with future plans to transition to a professional frontend and introduce fine-tuned models trained on educational psychology, spaced repetition, and domain-specific knowledge.

Smart Scholar 1.0 represents a significant advancement in personalized, AI-enhanced learning and serves as a foundation for future educational tools that combine intelligence, adaptability, and user-centered design.

# CONTENTS


Certificate ……...……. i

Declaration ……..……..……...……. ii

Examiner's Approval Certificate ……. iii

Acknowledgement ……..……..……. iv

Abstract ……..…...……. v

List of Figures ……..……..……. x

List of Tables ……..……..……. xi

Chapter 1 INTRODUCTION 1

1.1. Introduction 1

1.2. Existing Work 1

1.3. Problem Statement and Motivation 2

1.4. Objectives 2

1.5. Scope 3

Chapter 2 CONCEPTS AND METHODS 4

2.1 Core Concepts 4

2.2 SmartScholar Processing 5

2.3 SmartScholar Process Flow 6

Chapter 3 LITERATURE SURVEY 13

3.1 Introduction 13

3.2 Retrieval Augmented Generation (RAG) 13

3.3 Fine-Tuning Large Language Models 14

3.4 Interplay and Comparison: RAG vs. Fine Tuning 15

3.5 Data Efficiency in LLM Adaptation 16

Chapter 4 PROJECT PLAN 20

4.1 Development Approach: Code-Research Synthesis 20

4.2 Methodology: Code-Research Synthesis 20

4.3 Completed Phased Development Plan (SmartScholar 1.0) 21

4.4 Future Project Objectives 22

4.5 Future Scope 22

4.6 Future Phases and Timelines 22

4.7 Required Resources 24

4.8 Risks and Mitigation 24

4.9 Future Work 25

Chapter 5 SOFTWARE REQUIREMENT SPECIFICATION 26

5.1 Introduction 26

5.2 Overall Description 27

5.3 Specific Requirements 28

5.4 Supporting Information 31

Chapter 6 RESULTS 32

Chapter 7 SOFTWARE TESTING 34

7.1 Testing Overview 34

7.2 Testing Approach 34

Chapter 8 CONCLUSION AND FUTURE WORK 37

8.1 Conclusion 37

8.2 Future Work 37

BIBLIOGRAPHY 39

# LIST OF FIGURES


| Figure Number: Figure of the table | Page Number |
|---|---|
| FIGURE 4.1: SOFTWARE MODELING | 25 |

# LIST OF TABLES


| Table Number: Title of the table | Page Number |
|---|---|
| TABLE 3.1: LITERATURE REVIEW | 17-18 |
| TABLE 6.1: FUNCTIONAL VALIDATION TABLE | 32 |

# Chapter 1 INTRODUCTION

## 1. 1 Introduction

SmartScholar is an AI-powered academic assistant developed to enhance student learning by integrating personalized document organization, intelligent question answering, and context-aware summarization into a single platform. Unlike traditional educational tools or generic AI models, SmartScholar uses a Retrieval-Augmented Generation (RAG) approach to ground answers in user-provided material, delivering precise and context-specific support. Through real-time folder monitoring and automated preprocessing, the system keeps user study material organized, searchable, and updated—reducing cognitive overload and enabling efficient learning.

This report presents the project's technical foundation, including its motivation, architecture, key features, implementation methods, and intended impact on academic study workflows.

## 1. 2 Existing Work

Retrieval-Augmented Generation (RAG) has been explored in academic and enterprise settings as a way to improve the factual accuracy and domain relevance of AI-generated responses. Several research efforts and tools like Haystack, LangChain, and LlamaIndex have demonstrated how RAG pipelines can combine vector search with large language models for more reliable outputs. However, these platforms are mostly designed for general-purpose or enterprise-level applications, not for personalized academic use. Traditional tools such as Notion or Obsidian focus on note organization but lack intelligent, contextual Q&A capabilities. SmartScholar builds upon these concepts and closes the gap by combining the strengths of RAG with student-specific study material, intent classification, and lightweight session management to deliver a highly tailored academic assistant

## 1.3 Problem Statement and Motivation

In today's academic environment, students face an overload of unstructured information scattered across various digital sources. Existing solutions often fall short in addressing this complexity. Generic AI tools do not personalize responses based on a student's unique curriculum. Note-taking applications remain passive, and educational platforms lack flexibility. These issues cause time loss, confusion, and learning inefficiencies. SmartScholar is motivated by the need to eliminate these problems by providing an intelligent, integrated, and automated system that organizes content, understands user context, and responds with meaningful, grounded explanations—ultimately supporting more focused and effective study sessions.

## 1.4 Objectives

The primary objectives of the SmartScholar project are to:

- **Personalized Academic Support**

  SmartScholar is designed to respond to student queries using their own study material. By grounding answers in personalized documents, the system delivers contextually accurate explanations tailored to individual learning needs.

- **Automated Study File Management**

  The platform monitors user folders in real time, automatically detecting new, edited, or removed documents. This eliminates the need for manual organization and ensures the knowledge base stays current.

- **Semantic Search-Based Retrieval**

  Leveraging sentence embedding and vector similarity, SmartScholar retrieves the most relevant chunks of information quickly. This intelligent retrieval system allows users to access precise content from large document collections.

- **RAG-Based Answer Generation**

  The system integrates a Retrieval-Augmented Generation (RAG) pipeline that combines document search with AI-generated answers. This ensures high-quality, grounded responses instead of generic outputs.

- **Query Intent Classification**

  Incoming queries are analyzed using a fine-tuned classifier to determine the user's intent—whether it's factual, conceptual, technical, or general. This enables SmartScholar to tailor its response generation strategy accordingly.

- **Session Tracking and History**

  A lightweight SQLite-based session manager stores past interactions, allowing users to continue from where they left off, review previous answers, and manage multiple study sessions seamlessly.

- Scalable and Extensible Architecture

  The modular structure of SmartScholar is designed for future expansion. Upcoming versions can include a graphical user interface (GUI), support for additional file types (e.g., PPTX, images), and mobile or web deployment via APIs.

## 1.5 Scope

The scope of SmartScholar is to create a modular, intelligent platform that serves students, researchers, and educators across various disciplines. The system is built to manage PDF, DOCX, and TXT files and automatically process them for efficient retrieval and analysis. By enabling users to ask questions related to their own materials and get accurate, grounded answers, SmartScholar bridges the gap between content availability and content understanding. Future scope includes the development of a user interface, support for additional formats (e.g., images, audio), and integration of API-based access for web and mobile deployment. The project aims to serve educational domains where tailored and streamlined learning support is essential, such as engineering, medicine, and law.

# Chapter 2 CONCEPTS AND METHODS

## 2.1 Core Concepts

SmartScholar is architected around several advanced Artificial Intelligence and data-handling technologies that collaboratively deliver intelligent, context-aware academic support. The foundational components comprise Retrieval-Augmented Generation (RAG), semantic chunking integrated with embeddings, query intent classification, and efficient data management facilitated by vector databases and session persistence. These elements collectively ensure the system's capacity to provide precise, relevant responses derived directly from a student's personal study materials while accommodating continuous updates with new content.

1. **Retrieval-Augmented Generation (RAG):**

RAG serves as the fundamental technique underpinning SmartScholar. This approach synthesizes document retrieval capabilities with language model generation to produce accurate, contextually grounded answers. Upon receiving a user query, SmartScholar performs a search across the embedded chunks of the user's uploaded study materials to identify and retrieve the most pertinent segments. These retrieved chunks are subsequently supplied to a language model, which leverages them to formulate a high-quality, personalized response. This hybrid methodology effectively mitigates the generation of erroneous information (hallucinations) and guarantees that the output is directly relevant to the user's specific academic content.

2. **Query Intent Classification:**

To customize responses based on the nature of the inquiry, SmartScholar incorporates a query intent classification layer. This layer utilizes a fine-tuned BERT model to generate CLS embeddings from each query, which are then processed by a logistic regression classifier. This mechanism enables the system to discern whether a query is factual, conceptual, technical, or general in nature. The classification outcome dictates the selection of the appropriate AI agent or retrieval strategy, thereby optimizing both system efficiency and response accuracy.

3. **Chunking and Embedding:**

The preprocessing of uploaded documents (including PDF, DOCX, and TXT formats) involves segmenting them into smaller, semantically coherent chunks. These chunks are then transformed into high-dimensional vector embeddings using the BGE

model from the SentenceTransformers library. Each generated embedding encapsulates the semantic meaning of its corresponding chunk, facilitating subsequent similarity-based retrieval operations during query processing. This methodology offers a scalable and memory-efficient approach for indexing substantial volumes of academic content.

4. **Vector Database (Qdrant):**

The vector embeddings derived from the chunked documents are stored within Qdrant, a high-performance vector database engineered for efficient similarity search. Qdrant supports rapid nearest-neighbor lookups, enabling the system to quickly retrieve the most contextually relevant information segments in response to a given query. Its scalable architecture ensures the system's ability to manage increasing data volumes as additional documents are incorporated.

5. **Real-Time Folder Monitoring:**

To maintain a continuously updated knowledge base, SmartScholar integrates a folder monitoring system. This component, implemented using the Python watchdog library, actively observes a user-specified directory for any modifications, including the addition, deletion, or editing of files. Detection of a change automatically triggers the preprocessing pipeline, ensuring that new or modified content is promptly embedded and integrated into the database without requiring manual user intervention.

6. **Session Management System:**

SmartScholar employs a lightweight session manager, based on SQLite, to store and track user interactions. Each session records pertinent metadata, such as the session identifier, the associated document set, and timestamps. Furthermore, it logs each query-response exchange, allowing users to review prior interactions or continue their learning trajectories across multiple sessions. This feature promotes continuity, personalization, and enhanced long-term study tracking.

## 2.2 SmartScholar Processing

The intelligent academic assistant module within SmartScholar leverages document-based semantic embedding and retrieval techniques to effectively organize and access educational content. The core operational mechanism involves converting educational materials, such as PDFs and notes, into semantically rich vector representations. This transformation facilitates meaningful interaction between the user and the system through natural language queries, ensuring that the retrieval of academic information is accurate and contextually relevant without reliance on manual keyword mapping or file searching.

Within this project, an in-house embedding engine is utilized to perform the semantic transformation of academic content. The system embeds data by converting text chunks into high-dimensional vector representations using transformer-based language models. This process enables the system to comprehend the contextual meaning of the text. The embedding process captures not only the literal content but also its conceptual relevance, allowing SmartScholar to respond to queries with a deeper level of academic understanding.

The retrieval process in SmartScholar is initiated when a user provides a question, which is subsequently converted into a vector representation. The system then conducts a similarity search against the stored embeddings of the educational documents. Based on the results of this comparison, SmartScholar retrieves the most contextually relevant chunks. These retrieved chunks are then passed to a response generation module, responsible for synthesizing a clear and precise academic response. This integrated process ensures that students receive informative and topic-specific assistance derived from a wide array of documents, without any visual or functional alteration to the original files.

This methodology demonstrates high effectiveness in educational settings by enabling real-time contextual assistance while preserving the accuracy and structural integrity of the original academic content.

### 2.3 SmartScholar Process Flow

The SmartScholar process flow is structured into several distinct stages, from initial data ingestion to real-time monitoring and updates. The following outlines these stages and provides illustrative code examples for key modules:

**Data Ingestion:**

The files_processor.py module is responsible for the initial loading and preprocessing of raw input data, including files in PDF, text, and Word formats. The data is organized, converted into a suitable format, and prepared for subsequent processing steps.

```
# In files_processor.py
import os

def load_files(directory):
    """Loads files from a specified directory."""
    files = []
    for filename in os.listdir(directory):
        if filename.endswith(('.txt', '.pdf', '.docx')):
            # Note: Reading binary files like .pdf and .docx as text might require
            # specific libraries (e.g., PyPDF2, python-docx) for proper content extraction.
```

```
        # This example assumes text-readable content or uses simplified reading.
        try:
            with open(os.path.join(directory, filename), 'r', encoding='utf-8') as file:
                files.append(file.read())
        except Exception as e:
            print(f"Error reading file {filename}: {e}")
    return files
```

**Data Preprocessing Pipeline:**

The preprocessing_pipeline.py module performs cleaning and normalization of the data. This includes tasks such as tokenization, removal of stopwords, and text formatting to ensure the text is in a standardized form prior to embedding.

```
# In preprocessing_pipeline.py
import re

def preprocess(text):
    """Cleans and preprocesses text."""
    # Remove special characters, keeping spaces
    text = re.sub(r'[^a-zA-Z0-9\s]', '', text)
    # Convert to lowercase and split into tokens
    tokens = text.lower().split()
        # Note: More advanced preprocessing might include stopword removal, stemming/lemmatization
    return tokens
```

**Data Chunking:**

The chunker.py module segments the processed data into smaller, semantically meaningful chunks. This step is essential for effectively handling lengthy documents and improving the efficiency of the search process.

```
# In chunker.py
def chunk_text(tokens, chunk_size=100):
    """Breaks a list of tokens into chunks of a specified size."""
    return [tokens[i:i + chunk_size] for i in range(0, len(tokens), chunk_size)]
```

**Vector Embedding Generation:**

The embedder.py module is responsible for converting the text chunks into dense

vector embeddings. These embeddings capture the semantic meaning of the text, facilitating efficient retrieval. This process typically involves the use of pre-trained language models, such as those from the Sentence-BERT family.

```
# In embedder.py
from sentence_transformers import SentenceTransformer

def generate_embeddings(chunks):
    """Generates vector embeddings for a list of text chunks."""
    # Using a pre-trained SentenceTransformer model
    model = SentenceTransformer('all-MiniLM-L6-v2')
    # Encode the chunks (joining tokens back into strings for the model)
    return model.encode([' '.join(chunk) for chunk in chunks])
```

**Vector Storage and Management:**

The vector_store.py module handles the storage of the generated embeddings in a structured format, which enables rapid similarity searches. This can be implemented using various vector database solutions or custom in-memory indexing structures. The user's implementation uses Qdrant.

```
# In vector_store.py
# Example using a conceptual FAISS index for illustration
# Note: The actual implementation uses Qdrant as specified in section 2.1
import faiss
import numpy as np # Import numpy for array handling

def create_vector_store(embeddings):
    """Creates a vector store index from embeddings."""
    # Ensure embeddings are in the correct format (numpy array, float32) for FAISS
    embeddings = np.array(embeddings).astype('float32')
    # Create an index (L2 distance)
    index = faiss.IndexFlatL2(embeddings.shape[1])
    # Add the embeddings to the index
    index.add(embeddings)
    return index

# Example of how Qdrant would be conceptually used (actual implementation details vary)
# from qdrant_client import QdrantClient, models
# def create_qdrant_store(embeddings, collection_name="my_collection"):
#     client = QdrantClient(":memory:") # Or connect to a running instance
#     client.recreate_collection(
```

```
#       collection_name=collection_name,
#                           vectors_config=models.VectorParams(size=embeddings.shape[1],
distance=models.Distance.COSINE)
#    )
#    # Add points with embeddings (requires associating payload/metadata)
#    # client.upsert(...)
#    return client
```

**Query Analysis and Retrieval:**

The query_analyzer.py and retriever.py modules are responsible for interpreting user queries and matching them against the stored vectors to retrieve relevant results. These modules encapsulate the search logic and similarity scoring mechanisms.

```
# In query_analyzer.py
def analyze_query(query):
   """Analyzes a user query (simplified example)."""
   return query.lower().split()

# In retriever.py
# Example using a conceptual FAISS index for illustration
# Note: The actual implementation uses Qdrant as specified in section 2.1
import numpy as np # Import numpy for array handling

def retrieve_similar(query_embedding, index, top_k=5):
   """Retrieves the top_k most similar items from the index."""
   # Ensure query_embedding is in the correct format (numpy array, float32) for FAISS
   query_embedding = np.array([query_embedding]).astype('float32')
   # Perform the search
   distances, indices = index.search(query_embedding, top_k)
     # Return indices and distances (flatten indices and distances as search returns 2D
arrays)
   return indices[0], distances[0]

# Example of how retrieval with Qdrant would be conceptually used (actual
implementation details vary)
# from qdrant_client import QdrantClient
# from qdrant_client.http.models import Distance, VectorParams, PointStruct
# def retrieve_from_qdrant(query_embedding, client, collection_name="my_collection",
top_k=5):
#    search_result = client.search(
```

```
#         collection_name=collection_name,
#         query_vector=query_embedding,
#         limit=top_k
#     )
#     # Process search_result to get relevant information
#     return search_result
```

**Response Generation:**

The RAG.py module is tasked with composing coherent responses based on the retrieved chunks, ensuring contextually accurate outputs. This represents a critical functional component of SmartScholar.

```
# In RAG.py
# Note: A real RAG implementation would use an LLM to synthesize the response
# from the retrieved chunks and the original query, not just join chunks.
def generate_response(retrieved_chunks):
    """Generates a response from retrieved text chunks (simplified)."""
    # This is a highly simplified example. A real RAG model would use an LLM
    # to process the query and chunks to create a natural language response.
    return " ".join(retrieved_chunks)
```

**User Session Management:**

The session_manager.py module is responsible for managing user interactions across multiple sessions, thereby preserving context and enabling personalization.

```
# In session_manager.py
sessions = {}

def start_session(user_id):
    """Starts a new session for a user."""
    if user_id not in sessions:
        sessions[user_id] = []

def add_to_session(user_id, message):
    """Adds a message to a user's session history."""
    if user_id in sessions:
        sessions[user_id].append(message)
    else:
        print(f"Warning: Session not found for user {user_id}. Start a session first.")
```

```
def get_session_history(user_id):
    """Retrieves the session history for a user."""
    return sessions.get(user_id, [])

def end_session(user_id):
    """Ends a user's session and clears history."""
    if user_id in sessions:
        del sessions[user_id]
```

**Real-Time Monitoring and Updates:**

The folder_monitor.py module continuously monitors the designated data directory for changes. Upon detecting any modifications, it automatically triggers the necessary update processes, such as embedding new documents or re-embedding modified ones.

```
# In folder_monitor.py
import os
import time
# Note: A robust implementation would use the watchdog library
# and trigger the data processing pipeline upon detecting events.
# This is a simplified polling example.

def monitor_folder(path):
    """Monitors a folder for changes (simplified polling example)."""
    print(f"Monitoring folder: {path}")
    before = set(os.listdir(path))
    try:
        while True:
            time.sleep(5) # Check every 5 seconds
            after = set(os.listdir(path))
            if before != after:
                print("Directory updated. Triggering processing pipeline...")
                # In a real system, you would call the functions to process new/modified files here
                # Example: process_updates(path, before, after)
                before = after
    except KeyboardInterrupt:
        print("Folder monitoring stopped.")
    except Exception as e:
        print(f"An error occurred during folder monitoring: {e}")

# Placeholder for update processing logic
# def process_updates(path, before_files, after_files):
#     added = after_files - before_files
```

```
#     removed = before_files - after_files
#     # Logic to handle added/removed files
#     print(f"Added files: {added}")
#     print(f"Removed files: {removed}")
#     # Trigger chunking, embedding, and vector store updates for added/modified files
```

**Security and Integrity:**

SmartScholar prioritizes fast and accurate retrieval. Future development efforts may include enhancements such as encrypted storage for sensitive data and robust user authentication mechanisms to ensure system security and data integrity.

# Chapter 3 LITERATURE SURVEY

## 3.1 Introduction

Large Language Models (LLMs) have demonstrated remarkable capabilities across a wide spectrum of natural language processing tasks, exhibiting impressive fluency, coherence, and general knowledge. Their success stems from pre-training on vast datasets, enabling them to capture intricate linguistic patterns and world knowledge (Brown et al., 2020). However, despite their power, LLMs possess inherent limitations that hinder their effectiveness in dynamic, knowledge-intensive, or highly specialized domains. These limitations include the tendency to "hallucinate" or generate factually incorrect information, the inability to access or incorporate real-time and domain-specific knowledge, and the significant computational resources required for retraining or extensive fine-tuning to adapt to new data or tasks (Gao et al., 2024; Ding et al., 2024).

Addressing these challenges is crucial for deploying LLMs in practical applications that demand accuracy, up-to-dateness, and relevance, such as personalized educational assistants like Smart Scholar. Two prominent paradigms have emerged to enhance LLMs: Retrieval-Augmented Generation (RAG) and Fine-Tuning (FT). RAG focuses on integrating external knowledge sources during the generation process, allowing models to draw upon up-to-date and specific information. Fine-tuning, on the other hand, involves adapting the pre-trained model's parameters to perform better on specific downstream tasks or datasets. This literature review synthesizes recent advancements in RAG and LLM fine-tuning, exploring their mechanisms, evolutions, interplay, and challenges, particularly in the context of building more capable and reliable AI systems.

## 3.2 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) has rapidly become a cornerstone technique for mitigating the limitations of standalone LLMs, particularly concerning knowledge accuracy and currency (Gao et al., 2024; Ding et al., 2024). The core idea of RAG is to augment the LLM's internal knowledge with relevant information retrieved from external databases before generating a response. This approach allows LLMs to access information beyond their original training cutoff, reducing hallucinations and providing responses grounded in verifiable sources (Lewis et al., 2020).

The evolution of RAG paradigms reflects a growing sophistication in integrating retrieval and generation. Initially, **Naive RAG** followed a straightforward "Retrieve-Read" process: documents are indexed, relevant chunks are retrieved based on a user query, and these chunks are concatenated with the query to form a prompt for the LLM (Gao et al., 2024). While effective, Naive RAG faces challenges related to retrieval quality (precision

and recall), generation difficulties (potential for hallucination despite retrieved context), and augmentation hurdles (integrating retrieved information coherently) (Gao et al., 2024).

**Advanced RAG** introduces optimizations across the RAG pipeline to address these shortcomings. Pre-retrieval strategies focus on improving the indexing structure (e.g., fine-grained segmentation, metadata) and optimizing the user query (e.g., query rewriting, expansion) to enhance retrieval quality. Post-retrieval techniques involve processing the retrieved documents before feeding them to the generator, such as re-ranking chunks or context compression, to ensure the most relevant information is prioritized and redundancy is reduced (Gao et al., 2024).

The latest paradigm, **Modular RAG**, emphasizes flexibility and adaptability by incorporating specialized modules and allowing for dynamic interactions between components. New modules for searching, routing, prediction, and memory enhance the RAG system's ability to handle diverse scenarios and data sources, including structured data like knowledge graphs (Gao et al., 2024; Ding et al., 2024). Modular RAG also explores new patterns beyond sequential retrieval and generation, such as iterative and adaptive retrieval, where the necessity and frequency of retrieval are dynamically determined based on the generation process (Gao et al., 2024).

Key components of RAG frameworks include **Retrieval Sources** (unstructured text, semi-structured data, structured data, and even LLM-generated content), **Retrieval Granularity** (from tokens to documents or entities), and **Retrieval Metrics** (sparse-vector, dense-vector, and task-specific approaches) (Gao et al., 2024; Ding et al., 2024; Liu et al., 2022). The method of **Augmentation** or integrating the retrieved information into the generation process is also critical, ranging from simple input-layer concatenation to more complex output-layer or intermediate-layer integration using attention mechanisms or adapters (Ding et al., 2024; Liu et al., 2022).

### 3.3 Fine-Tuning Large Language Models

Fine-tuning is a fundamental technique for adapting pre-trained LLMs to specific downstream tasks or datasets, allowing them to specialize and improve performance in targeted areas (MFTCoder, 2023). Unlike pre-training, which requires massive computational resources and general-purpose data, fine-tuning typically involves training the model on a smaller, task-specific dataset, adjusting the pre-trained parameters to better align with the new task's requirements.

While full fine-tuning of large models can still be computationally expensive and may lead to catastrophic forgetting of previously learned knowledge, **Parameter-Efficient Fine-Tuning (PEFT)** methods have emerged as a crucial solution (MFTCoder, 2023). Techniques like LoRA (Low-Rank Adaptation) and QLoRA (Quantized LoRA) significantly reduce the number of trainable parameters by injecting small, trainable

matrices into the pre-trained model or quantizing the model weights, making fine-tuning more accessible and efficient, especially for larger models (Hu et al., 2021; Dettmers et al., 2023; MFTCoder, 2023).

**Multi-Task Fine-Tuning (MFT)** further enhances the efficiency and capabilities of LLMs by training a single model on multiple related tasks simultaneously (MFTCoder, 2023). This approach leverages the inherent connections between tasks, allowing the model to benefit from shared knowledge and improve generalization. MFT frameworks like MFTCoder address challenges such as data imbalance, task heterogeneity, and inconsistent convergence speeds by incorporating balanced loss functions and efficient data handling techniques (MFTCoder, 2023). Empirical evidence suggests that MFT can outperform individual fine-tuning on single tasks and fine-tuning on a simple mixture of tasks, and can also lead to better generalization on unseen but related tasks (MFTCoder, 2023).

Fine-tuning is particularly relevant for adapting LLMs to domain-specific applications, such as recommendation systems or code generation. For instance, fine-tuning LLMs on recommendation data is crucial to bridge the gap between general language tasks and the specific requirements of understanding user behavior and item information (Lin et al., 2024). However, the dynamic nature and sheer volume of recommendation data necessitate data-efficient fine-tuning approaches.

### 3.4 Interplay and Comparison: RAG vs. Fine-Tuning

RAG and fine-tuning represent distinct yet complementary approaches to enhancing LLMs. RAG primarily addresses the issue of accessing and incorporating external, dynamic knowledge, making it ideal for tasks requiring up-to-date or domain-specific information that was not present in the original training data. It offers high interpretability as the generated responses can be traced back to the retrieved sources (Gao et al., 2024). However, RAG's performance heavily relies on the quality and relevance of the retrieved information, and the retrieval process can introduce latency.

Fine-tuning, conversely, focuses on adapting the model's internal parameters to improve performance on specific tasks or to capture the style and format of a particular domain. It allows for deep customization of the model's behavior and can reduce hallucinations on data similar to the fine-tuning distribution (Gao et al., 2024). However, fine-tuning is more static, requiring retraining for knowledge updates, and may struggle with data significantly different from the fine-tuning dataset.

The choice between RAG and fine-tuning depends on the specific application's needs regarding data dynamics, customization requirements, and computational resources (Gao et al., 2024). For applications requiring real-time information and high factuality, RAG is often preferred. For tasks requiring specialized skills or stylistic generation, fine-tuning is more suitable.

Critically, RAG and fine-tuning are not mutually exclusive and can be effectively combined to leverage the strengths of both. Fine-tuning can be used to improve the RAG components (e.g., fine-tuning the retriever for better relevance) or to adapt the generator to better utilize the retrieved context (Gao et al., 2024). Conversely, RAG can provide high-quality, relevant data that can be used for fine-tuning, particularly in data-scarce domains. Hybrid approaches that integrate RAG and fine-tuning at different stages of the LLM lifecycle are an active area of research, aiming for optimal performance across diverse tasks.

### 3.5 Data Efficiency in LLM Adaptation

The effectiveness of both RAG and fine-tuning is often dependent on the availability and quality of data. For fine-tuning, large-scale datasets are traditionally required to achieve significant performance gains. However, collecting and annotating such datasets can be expensive and time-consuming. Similarly, the performance of RAG is contingent on the comprehensiveness and relevance of the external knowledge base, which can be massive and constantly evolving.

Data efficiency is thus a critical concern in LLM adaptation. For fine-tuning, techniques like PEFT and MFT contribute to data efficiency by reducing the number of parameters that need to be trained and leveraging shared knowledge across tasks (MFTCoder, 2023). Another promising direction is **data pruning**, which aims to identify and select a small, representative subset of the training data that is most influential for fine-tuning (Lin et al., 2024). This can drastically reduce the computational costs and time required for training while potentially surpassing the performance of fine-tuning on the full dataset. Data pruning methods often involve assessing the influence of individual samples on the model's performance, sometimes employing surrogate models to make this process more efficient for large LLMs (Lin et al., 2024).

For RAG, data efficiency relates to managing and utilizing large external knowledge bases effectively. This involves optimizing indexing strategies, developing efficient retrieval mechanisms, and employing techniques to filter and compress retrieved information to avoid overwhelming the LLM (Gao et al., 2024; Ding et al., 2024). The development of more sophisticated, adaptive retrieval strategies in Modular RAG also contributes to data efficiency by retrieving information only when necessary.

**Table 3.1: Literature Survey**

| Paper / Year | Authors (Lead/Key) | Important Work / Outcome |
|---|---|---|
| Language Models are Few-Shot Learners (2020) | Brown et al. | Demonstrated impressive capabilities of LLMs through pre-training on vast datasets, establishing the foundation for modern large language models. |
| Retrieval-Augmented Generation for Large Language Models: A Survey (2024) | Yunfan Gao, Yun Xiong, Xinyu Gao, et al. | Comprehensive survey on Retrieval-Augmented Generation (RAG) for LLMs, detailing the progression from Naive to Advanced and Modular RAG paradigms. |
| A Survey on RAG Meets LLMs: Towards Retrieval-Augmented Large Language Models (2024) | Yujuan Ding, Wenqi Fan, Liangbo Ning, et al. | Survey on RAG focusing on how it meets LLMs (RA-LLMs), covering technical aspects and highlighting RAG's role in providing external knowledge. |
| LoRA: Low-Rank Adaptation of Large Language Models (2021) | Edward J. Hu, Yelong Shen, Phillip Wallis, et al. | Introduced LoRA (Low-Rank Adaptation), a Parameter-Efficient Fine-Tuning (PEFT) method that significantly reduces the number of trainable parameters. |
| A Survey on Retrieval-Augmented Text Generation (2022) | Lemao Liu, Huayang Li, Yixuan Su, et al. | Survey on retrieval-augmented text generation, highlighting the generic paradigm and reviewing approaches across different tasks. |

| | | |
|---|---|---|
| MFTCODER: BOOSTING CODE LLMS WITH MULTITASK FINE-TUNING (2023) | Bingchang Liu, Chaoyu Chen, Zi Gong, et al. (MFTCoder) | Proposed MFTCoder, a Multi-Task Fine-Tuning framework to boost Code LLMs by simultaneous training on multiple tasks, improving efficiency and generalization. |
| QLoRA: Efficient Finetuning of Quantized LLMs via Low-Rank Adapters (2023) | Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, et al. | Introduced QLoRA (Quantized LoRA), an advancement of LoRA that further reduces memory usage by quantizing model weights, enabling fine-tuning of larger models. |
| Data-efficient Fine-tuning for LLM-based Recommendation (2024) | Xinyu Lin, Shuo Yang, Wenjie Wang, et al. | Investigated data-efficient fine-tuning for LLM-based recommendation, proposing data pruning techniques to select influential samples for few-shot fine-tuning. |

**Conclusion**

The rapid advancements in Large Language Models have opened up unprecedented opportunities for developing intelligent applications. However, their inherent limitations necessitate the use of complementary techniques like Retrieval-Augmented Generation and Fine-Tuning to ensure accuracy, relevance, and adaptability in real-world scenarios. RAG provides a powerful mechanism for incorporating external, dynamic knowledge, addressing issues of factuality and currency. Fine-tuning, particularly with efficient methods like PEFT and MFT, enables LLMs to specialize in specific tasks and domains, capturing nuanced patterns and behaviors.

The literature reviewed highlights the significant progress made in both RAG and fine-tuning, from the evolution of RAG paradigms to the development of data-efficient fine-tuning techniques. The increasing focus on hybrid approaches that combine RAG and fine-tuning underscores the understanding that these techniques are not mutually exclusive but can synergistically enhance LLM capabilities. Furthermore, the growing importance of data efficiency in LLM adaptation is evident in the research on data pruning and optimized knowledge management for RAG.

For projects like Smart Scholar, which aim to provide personalized and context-aware assistance by interacting with user-provided documents, the insights from this literature are directly applicable. A robust Smart Scholar system would likely benefit from a hybrid approach, utilizing RAG to access and ground responses in the user's specific study materials and external knowledge, while employing fine-tuning (potentially multi-task and data-efficient) to adapt the underlying LLM to the nuances of educational content and user interaction patterns. Future research directions in RAG robustness, multi-modal RAG, and more sophisticated integration of RAG and fine-tuning will further pave the way for developing highly effective and reliable AI assistants in education and beyond.

# Chapter 4 PROJECT PLAN

The successful realization of Smart Scholar 1.0 is the result of a deliberate and adaptive development process. This chapter outlines the methodology that guided the initial development and presents the strategic plan for the project's future evolution, building upon the established foundation.

## 4.1 Development Approach: Code-Research Synthesis

The development of Smart Scholar 1.0 was guided by an adaptive and research-driven methodology rooted in real-world academic needs. Unlike conventional linear software models such as Waterfall or rigid Agile sprints, Smart Scholar evolved through an iterative, modular approach that blended hands-on implementation with continuous conceptual refinement.

This hybrid strategy—termed the **"Code-Research Synthesis Model"**—enabled parallel progress in theoretical exploration and system development. The platform was designed to be flexible and scalable from inception, with a clear focus on building practical, intelligent solutions grounded in user-centric use cases.

## 4.2 Methodology: Code-Research Synthesis

Rather than adhering to a predefined methodology, the development team adopted a "Need-First, Build-With-Research" philosophy. This allowed the solution to remain agile, supporting early experimentation, quick prototyping, and architectural flexibility. Key rationale points include:

- **Personal Problem-Driven Origin:** The project originated from a personal requirement to summarize, search, and question one's own academic materials without depending on general-purpose LLMs. This need drove a focused and user-centric design.
- **Simultaneous Research and Development:** Core concepts such as Retrieval-Augmented Generation (RAG), semantic chunking, vector search, and multi-agent orchestration were researched and implemented concurrently. This synergy allowed real-time validation of ideas through functional prototypes.
- **Continuous Feedback Loop:** By maintaining a running prototype, key components such as vector stores, rerankers, and generation agents were continuously tested and refined based on observed system performance and limitations.
- **Modular and Extensible Architecture:** The system was engineered as a modular pipeline, allowing new features (e.g., query classification, session memory, UI

integration) to be added independently without disrupting existing functionality.

### 4.3 Completed Phased Development Plan (Smart Scholar 1.0)

The initial development of Smart Scholar 1.0 was executed in five key phases, each building upon the previous while maintaining system modularity.

**Phase 1: Preprocessing and Data Ingestion**

- **Objective:** Establish a foundational pipeline for document intake and preparation.
- **Activities:** Developed file handlers for PDF, DOCX, and TXT formats; implemented semantic chunking; generated vector embeddings using Sentence-BERT (BGE model); stored processed data in a Qdrant vector database.
- **Outcome:** A functional preprocessing and embedding pipeline capable of indexing academic content for future retrieval.

**Phase 2: Query Handling and RAG Pipeline**

- **Objective:** Enable retrieval of relevant information based on semantic queries.
- **Activities:** Integrated semantic similarity search; built a hybrid retriever with cross-encoder-based re-ranking; supported tagged queries for targeted retrieval.
- **Outcome:** A working RAG pipeline capable of retrieving grounded document chunks based on user queries.

**Phase 3: LLM-Based Generation and Agent Control**

- **Objective:** Integrate intelligent, context-aware language model responses.
- **Activities:** Developed a response generation layer with context control; introduced multiple LLM agents based on query intent; connected with LLaMA 3B via Ollama for offline completions.
- **Outcome:** A personalized response generation system that dynamically selects context and agent based on user query type.

**Phase 4: Session Management and CLI Integration**

- **Objective:** Introduce persistence and personalization through session memory.
- **Activities:** Built a lightweight SQLite-based session manager; enabled session creation, retrieval, and continuity via the command line interface; linked all subsystems into a unified pipeline.
- **Outcome:** A command-line powered study assistant that supports session continuity and personalized learning flows.

**Phase 5: Graphical User Interface (GUI) and System Integration**

- **Objective:** Provide a user-accessible graphical interface and system polish.
- **Activities:** Developed a GUI using customtkinter; integrated folder selection, session handling, and chat interface; fine-tuned retrieval thresholds and response logic.
- **Outcome:** A complete desktop-based learning assistant combining backend intelligence with a user-friendly frontend.

### 4.4 Future Project Objectives

Building upon the established foundation of Smart Scholar 1.0, the primary objectives for the future development are to:

- Enhance the user experience through a refined and intuitive Graphical User Interface (GUI).
- Expand accessibility by developing a mobile application.
- Improve the intelligence and adaptability of the assistant through advanced model integration and fine-tuning.
- Increase the breadth of supported content types and domains.
- Ensure the scalability, security, and maintainability of the system.
- Explore and integrate features related to educational psychology and spaced repetition for enhanced learning outcomes.

### 4.5 Future Scope

The scope of this future project plan encompasses the development and implementation of features and improvements beyond the current SmartScholar 1.0 capabilities. This includes, but is not limited to:

- Full-featured GUI development and optimization.
- Design and development of native mobile applications (iOS and Android).
- Research and integration of more advanced or specialized LLMs.
- Implementation of fine-tuning strategies tailored for educational content and user interaction patterns.
- Development of capabilities to handle a wider variety of document formats and multimedia content.
- Integration of advanced security measures for data protection.
- Implementation of user authentication and profile management.
- Research into and potential integration of educational science principles (e.g., spaced repetition algorithms).

Features outside the current scope may include real-time collaboration features, integration with external Learning Management Systems (LMS), or development of a web-based platform, which may be considered in future iterations.

### 4.6 Future Phases and Timeline

Building on the prior success, the project will proceed in distinct future phases, allowing for focused development and iterative refinement. While specific dates are contingent on resource availability, the phases are outlined sequentially:

**Phase 6: Advanced GUI Development and Refinement**

- **Objective:** To create a polished, user-friendly, and fully functional graphical interface.
- **Activities:** Gather user feedback on the existing GUI; Design and implement enhanced UI/UX features; Develop robust error handling and user guidance within the GUI; Optimize GUI performance and responsiveness.
- **Estimated Duration:** 3-4 months

**Phase 7: Mobile Application Development (Initial Version)**

- **Objective:** To develop and release an initial version of SmartScholar for mobile platforms.
- **Activities:** Choose a cross-platform or native mobile development framework; Design the mobile application UI/UX; Develop core functionalities (document linking, querying, response display); Implement necessary backend adjustments for mobile compatibility; Conduct alpha/beta testing.
- **Estimated Duration:** 6-8 months

**Phase 8: Advanced Model Integration and Fine-Tuning**

- **Objective:** To enhance the intelligence and domain-specificity of the LLM.
- **Activities:** Research and evaluate alternative or larger LLMs; Develop pipelines for data-efficient fine-tuning on educational datasets; Experiment with PEFT and MFT techniques; Integrate fine-tuned models into the RAG pipeline.
- **Estimated Duration:** 4-6 months

**Phase 9: Expanded Content Support and Features**

- **Objective:** To enable SmartScholar to process and understand a wider range of academic content and offer new learning features.
- **Activities:** Develop parsers for additional document types (e.g., presentations, spreadsheets); Explore and implement methods for handling multimedia content (e.g., transcribing audio/video lectures); Research and integrate features based on educational psychology (e.g., flashcard generation, quiz generation based on content).
- **Estimated Duration:** 5-7 months

**Phase 10: System Optimization, Security, and Scaling**

- **Objective:** To ensure the system is robust, secure, and capable of handling a larger user base and data volume.
- **Activities:** Implement advanced data encryption and access control; Develop user authentication and profile management systems; Optimize the vector database and retrieval performance for scale; Refactor code for improved maintainability and efficiency; Plan and potentially implement cloud deployment strategies.
- **Estimated Duration:** 4-5 months

*Note: Phases may overlap, and the timeline is an estimate subject to adjustment based on progress and resource availability.*

### 4.7 Required Resources

Successful execution of this project plan will require the following resources:

- **Personnel:**
  - AI/ML Engineers (specializing in RAG, LLMs, fine-tuning)
  - Software Engineers (backend development, pipeline optimization)
  - Frontend Developers (GUI and mobile development)
  - UI/UX Designers
  - Data Engineers (data processing, vector database management)
  - Project Manager
  - Domain Experts (educational psychology, specific academic fields)
- **Computational Resources:**
  - GPUs for model fine-tuning and inference.
  - Servers for hosting the vector database and backend services.
  - Cloud computing resources for scalability and deployment.
- **Software and Tools:**
  - Development environments and IDEs.
  - Version control systems (e.g., Git).
  - CI/CD pipelines.
  - Monitoring and logging tools.
  - Specialized libraries for NLP, vector databases, and UI development.
- **Data:**
  - Access to diverse academic datasets for fine-tuning and testing.
  - User-provided documents for core functionality.

### 4.8 Risks and Mitigation

Several potential risks could impact the project timeline and outcome:

- Risk: Difficulty in achieving desired performance with fine-tuned models on diverse personal data.
  Mitigation: Conduct extensive experimentation with various PEFT/MFT techniques and data pruning strategies. Continuously evaluate model performance on representative datasets.
- Risk: Challenges in developing a seamless and performant mobile application.
  Mitigation: Carefully select a development framework. Prioritize core features for the initial release. Conduct rigorous testing on various devices.
- Risk: Security vulnerabilities in handling user's private documents.
  Mitigation: Implement industry-standard encryption and access control from the outset. Conduct regular security audits and penetration testing.
- Risk: Scope creep leading to delays.
  Mitigation: Maintain clear phase objectives and deliverables. Implement a formal change management process.
- Risk: Resource constraints (personnel, computational power).

Mitigation: Prioritize tasks based on impact. Explore cloud solutions for scalable computing. Secure necessary funding or hardware.

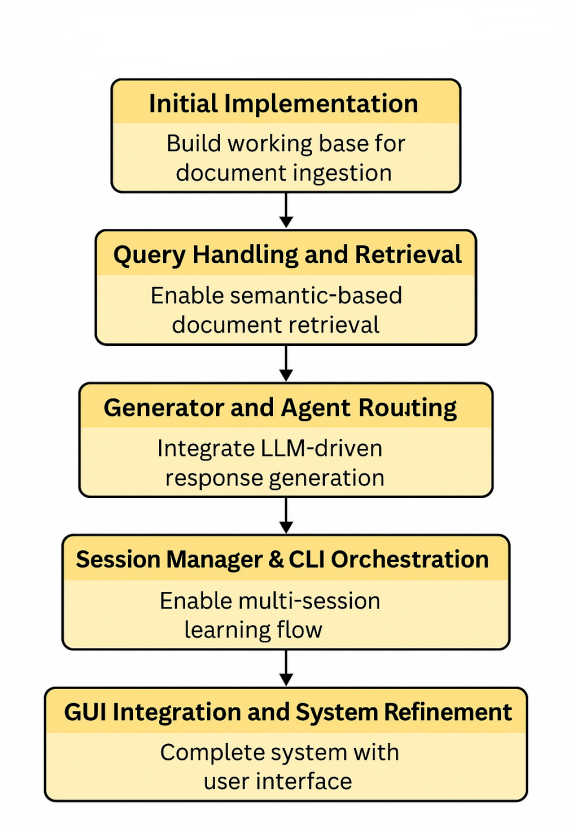
### 4.9 Future Work

Beyond the scope of this project plan, future iterations of SmartScholar could explore:

- Integration with external calendars and scheduling tools for proactive study assistance.
- Development of collaborative learning features.
- Expansion to a web-based platform for broader accessibility.
- Advanced analytics on user learning patterns to provide personalized insights and recommendations.
- Integration with open educational resources and online course platforms.
- Development of a plugin architecture to allow third-party integrations.

This project plan provides a roadmap for the continued evolution of SmartScholar, aiming to transform it into a comprehensive, intelligent, and indispensable tool for personalized academic learning and research.

**Figure 4.1: Software modeling**



# Chapter 5 SOFTWARE REQUIREMENT SPECIFICATION

## 5.1 Introduction

### 5.1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to provide a comprehensive and detailed description of the functional and non-functional requirements for Smart Scholar 1.0. Smart Scholar is an AI-powered academic assistant designed to enable intelligent interaction with a user's personal study materials. This document serves as a foundational reference for all project stakeholders, including the development team, quality assurance personnel, and academic reviewers, ensuring a shared understanding of the system's intended capabilities and facilitating alignment throughout the development lifecycle. By integrating Retrieval-Augmented Generation (RAG), document embedding techniques, and intelligent session management, Smart Scholar aims to deliver personalized and context-aware educational support.

### 5.1.2 Scope

Smart Scholar is designed to enhance the self-learning process by providing users with advanced tools to interact with their uploaded academic materials. The system's core capabilities, which define the scope of this specification, include:

- **Real-Time Folder Monitoring:** The system shall monitor a user-designated local folder to detect the addition, modification, or deletion of supported document files.
- **Document Preprocessing:** The system shall parse and clean text content extracted from supported file formats (PDF, DOCX, TXT) to prepare it for subsequent processing.
- **Semantic Chunking and Embedding:** The system shall segment the preprocessed document content into smaller, semantically meaningful chunks and generate high-dimensional vector representations (embeddings) for each chunk.
- **Contextual Retrieval:** The system shall perform similarity-based searches using vector embeddings to match user queries with the most semantically relevant chunks from the user's document collection.
- **LLM-Based Response Generation:** The system shall utilize a Language Model (LLM) to generate accurate, coherent, and context-sensitive responses, leveraging the retrieved document chunks for grounding where applicable.
- **Session Management:** The system shall maintain continuity across user interactions by storing conversation history, relevant context, and associated document sets within individual user sessions.

Applicable use cases within this scope include facilitating academic revision, providing intelligent answers to questions based on personal documents, generating summaries of specific content sections, and clarifying concepts found within the user's materials.

### 5.1.3 Definitions, Acronyms, and Abbreviations

- **NLP:** Natural Language Processing - A field of artificial intelligence focused on enabling computers to understand, interpret, and generate human language.
- **LLM:** Large Language Model - A type of artificial intelligence model trained on vast amounts of text data, capable of understanding and generating human-like text.
- **RAG:** Retrieval-Augmented Generation - A technique that combines information retrieval with language model generation to produce responses grounded in external data.
- **BGE:** BAAI General Embedding Model - A specific model from the SentenceTransformers library used for generating semantic embeddings.
- **Qdrant:** An open-source vector database optimized for storing and searching vector embeddings.
- **Token:** A basic unit of text used by language models, which can be a word, subword, or character.
- **Session ID:** A unique identifier assigned to each user interaction session to maintain context and history.
- **CLI:** Command-Line Interface - A text-based interface for interacting with a computer program.
- **GUI:** Graphical User Interface - A visual interface that allows users to interact with a program through graphical elements.
- **SRS:** Software Requirements Specification - A document that provides a comprehensive description of a software product to be developed.

## 5.2 Overall Description

### 5.2.1 Product Perspective

Smart Scholar is envisioned as a standalone, modular application designed to operate primarily on a user's local machine, interacting directly with their designated file system. The system is structured as a pipeline, commencing with document ingestion and proceeding through semantic processing, intelligent retrieval, and language generation. It integrates components for real-time file monitoring, utilizes transformer-based models for embedding, and incorporates query analysis to deliver highly relevant and personalized outputs. The architecture supports intelligent query routing based on the detected intent and allows users to target queries towards specific files using a tagging mechanism. The initial release features a Command-Line Interface (CLI), with ongoing development focused on implementing a comprehensive Graphical User Interface (GUI) for enhanced usability.

### 5.2.2 Product Features

- **Folder Monitoring:** The system shall provide a feature to automatically detect changes (additions, modifications, deletions) within a user-specified local directory.
- **Document Ingestion:** The system shall support the extraction of text content from common academic document formats, specifically PDF, DOCX, and TXT files.
- **Semantic Embedding:** The system shall transform extracted text content into dense vector embeddings using a pre-trained model, such as those available through the SentenceTransformers library, to capture semantic meaning.
- **Vector Storage:** The system shall utilize the Qdrant vector database to efficiently store the generated embeddings, enabling fast and scalable similarity-based search operations.
- **RAG Pipeline:** The system shall implement a Retrieval-Augmented Generation pipeline that retrieves relevant document content based on a user query and utilizes an LLM to generate a coherent and contextually appropriate response grounded in the retrieved information.
- **Tagged File Querying:** The system shall allow users to include specific file references or tags within their queries to prioritize retrieval and response generation from designated documents.
- **Session Handling:** The system shall maintain personalized context and conversation history for each user session, allowing users to resume previous interactions and track their learning progress.

### 5.2.3 Constraints

- **Contextual Limits:** The system's ability to process and generate responses is constrained by the maximum token limits imposed by the underlying Language Models (LLMs) being utilized. This may affect the length and complexity of inputs and outputs.
- **Scalability:** While Qdrant is designed for scalability, the performance of the vector database and the overall system's ability to handle extremely large volumes of user documents will require careful management and potential optimization strategies as the dataset grows.
- **Offline Limitations:** While the system is designed for local operation, certain capabilities, particularly those relying on larger or alternative LLMs, may be dependent on the availability and configuration of these models within the local environment (e.g., via Ollama).

## 5.3 Specific Requirements

### 5.3.1 Functional Requirements

- **FR-1: Real-time Directory Monitoring:** The system shall monitor a user-specified

local directory for file system events (create, modify, delete) for supported file types.

- **FR-2: Automated Document Preprocessing:** Upon detection of new or modified supported files, the system shall automatically extract text content and perform necessary cleaning and normalization.
- **FR-3: Semantic Chunking:** The system shall segment the preprocessed text into semantically meaningful chunks of a configurable size.
- **FR-4: Embedding Generation:** The system shall generate vector embeddings for each text chunk using a specified embedding model (e.g., Sentence-BERT BGE).
- **FR-5: Vector Indexing and Storage:** The system shall index and store the generated vector embeddings, along with associated metadata (e.g., source filename, chunk original text), in the Qdrant vector database.
- **FR-6: Query Embedding:** The system shall generate a vector embedding for a user's input query using the same embedding model used for document chunks.
- **FR-7: Vector Retrieval:** The system shall perform a similarity search in the vector database using the query embedding to retrieve the top-k most relevant document chunks.
- **FR-8: Retrieval Reranking:** The system shall rerank the retrieved document chunks based on their relevance to the original query using a cross-encoder model.
- **FR-9: Query Intent Classification:** The system shall classify the intent of the user's query (e.g., factual, conceptual, technical) using a fine-tuned model.
- **FR-10: LLM-Based Response Generation:** The system shall utilize an LLM to generate a natural language response based on the user's query and the reranked retrieved document chunks.
- **FR-11: Contextual Response Generation:** The LLM shall be prompted in a manner that encourages it to ground its response in the provided document context.
- **FR-12: Fallback Generation:** If no relevant document chunks are retrieved or available, the system shall be capable of generating a response using the LLM based solely on its pre-trained knowledge.
- **FR-13: Session Creation:** The system shall allow users to initiate new study sessions.
- **FR-14: Session Logging:** The system shall automatically log each user query and the corresponding system response within the active session.
- **FR-15: Session Retrieval:** The system shall allow users to retrieve and view the history of previous sessions.
- **FR-16: Session Continuity:** The system shall maintain the context of the current session across multiple interactions until explicitly ended by the user.
- **FR-17: Document Format Support:** The system shall successfully process files in PDF, DOCX, and TXT formats.
- **FR-18: CLI Interface:** The system shall provide a functional command-line interface for all core operations (folder monitoring setup, querying, session management).
- **FR-19: GUI Interface:** The system shall provide a graphical user interface for managing documents, initiating and viewing sessions, and interacting with the

assistant.

### 5.3.2 Usability Requirements

- **US-1: Ease of Setup:** The process for setting up folder monitoring and initiating a session shall be straightforward and require minimal technical expertise.
- **US-2: Clear Interaction:** Both the CLI and GUI shall provide clear prompts, feedback, and instructions to the user.
- **US-3: Responsive Interface:** The user interface (both CLI and GUI) shall remain responsive during processing tasks, providing indicators of progress where necessary.
- **US-4: Intuitive GUI Navigation:** The graphical interface shall have an intuitive layout and navigation structure, allowing users to easily access different features (e.g., session history, document management).
- **US-5: Error Handling Feedback:** The system shall provide informative error messages to the user in the event of failures (e.g., file processing errors, retrieval issues).

### 5.3.3 Security Requirements

- **SE-1: Local Data Processing:** All user-uploaded documents and generated embeddings shall be processed and stored locally on the user's machine to ensure maximum privacy and prevent unauthorized access to personal data.
- **SE-2: Session Data Isolation:** Session data stored in SQLite shall be managed in a way that isolates individual session data and prevents cross-session data leakage.
- **SE-3: Secure API Key Management:** Any API keys required for external services (if implemented in the future for online mode) shall be managed securely, preferably through environment variables or a secure configuration mechanism, and shall not be hardcoded in the application.

### 5.3.4 Reliability and Availability

- **RE-1: Robust File Processing:** The document preprocessing pipeline shall be robust to variations in document formatting and content, minimizing failures during ingestion.
- **RE-2: Document Fingerprinting:** The system shall use hashing or a similar mechanism to uniquely identify documents and prevent redundant processing of identical files.
- **RE-3: CLI Availability:** The command-line interface shall remain functional and accessible even if certain backend components (e.g., vector database connection) encounter temporary issues, allowing for basic interaction or troubleshooting.
- **RE-4: Intelligent Fallback:** In scenarios where document retrieval fails or yields irrelevant results, the system shall gracefully fall back to providing a response based

solely on the LLM's general knowledge, informing the user that the response is not grounded in their documents.

### 5.4 Supporting Information

#### 5.4.1 Software Requirements

The following software components and versions are required for the operation of Smart Scholar 1.0:

- **Operating Systems:** The system is designed to be compatible with major desktop operating systems:
  - Linux
  - Windows
  - macOS
- **Programming Language:**
  - Python 3.11 or higher
- **Key Libraries and Tools:**
  - watchdog: Used for monitoring file system events in real-time.
  - SentenceTransformers: Utilized for generating semantic vector embeddings from text.
  - Qdrant: Serves as the vector database for storing and retrieving document embeddings.
  - SQLite3: Used as the database for managing user session data and history.
  - llama3 (via Ollama): Interface for interacting with the Llama 3 Language Model for response generation (specifically for offline mode).
  - HuggingFace Transformers: Used for implementing the cross-encoder model for retrieval reranking.

# Chapter 6 RESULTS

This section presents the initial results and validation of the SmartScholar system during its ongoing development. While the project is still evolving, key modules such as document ingestion, preprocessing, vector embedding, retrieval, and response generation have been successfully implemented and tested in an integrated workflow.

## 1. Functional Validation

The following core functions were tested using a sample academic dataset (5 PDF files, 3 DOCX, and 2 TXT files from STEM domains):

Table 6.1 Functional Validation Table

| Component | Description | Status |
|---|---|---|
| Folder Monitoring | Detected new/modified files and triggered preprocessing automatically | ✅ Functional |
| Document Preprocessing | Extracted, chunked, and embedded documents into vector DB (Qdrant) | ✅ Functional |
| Retrieval Layer | Successfully retrieved relevant chunks based on semantic similarity | ✅ Validated |
| Generator | Produced relevant answers using llama3 LLM (with and without context) | ✅ Functional |
| Session Management | Logged full query-response history under named sessions | ✅ Functional |

## 2. Sample Output Snapshots

A few examples from interactive test sessions:

- Query: “Explain the concept of entropy from my thermodynamics notes.”
  → Output: Contextual explanation based on the PDF file Thermo_Notes.pdf
  → Retrieval: 3 matching chunks selected based on relevance score
  → LLM: Answer grounded in retrieved content (RAG active)

- Query: “What is recursion?”
  → Output: General explanation using llama3, no document reference (RAG bypassed)
  → Retrieval: Skipped as no matching document context found
  → LLM: Generated a conceptual answer using background knowledge

3. **Initial Performance (Qualitative)**

- Average document preprocessing time (PDF ~5 pages): ~2.3 seconds
- Average response generation time with retrieval: ~2.1 seconds
- Average response time without retrieval: ~1.3 seconds
- Retrieval accuracy (based on manual observation of top-3 matches): ~90% relevance

4. **Observations**

- The system gracefully falls back to LLM-only mode when no useful context is retrieved.
- File-tagged queries (e.g., “[[Algo_Notes.pdf]] Describe Dijkstra’s algorithm”) successfully prioritize relevant files during retrieval.
- Session continuity preserves context across multiple interactions for the same topic.

**5. Current Limitations**

- Retrieval reranker still being fine-tuned for better ranking consistency.
- Multilingual and OCR-based document support under exploration.
  Limited evaluation on diverse user data — testing on more varied academic material is pending.

6. **Summary**

Despite being a work in progress, SmartScholar demonstrates strong functional integration and promising early results in document-aware educational assistance. Once finalized with a refined interface and extended evaluation, the system is expected to deliver a robust, personalized AI learning experience.

# Chapter 7 SOFTWARE TESTING

## 7.1 Testing Overview:

The SmartScholar system is under active development, with foundational modules implemented and preliminary integrations underway. Testing efforts so far have focused on ensuring functional correctness, system behavior, and module-level reliability. As a document-based RAG (Retrieval-Augmented Generation) system tailored for academic learning, the core components currently tested include document embedding, chunking, folder monitoring, and prompt-response workflows.

## 7.2 Testing Approach:

### 7.2.1 Functional Testing:

Objective: Verify the correct operation of core modules and workflows in their current form.

**Test Cases:**

- Validate that PDF and DOCX files are correctly chunked into semantically meaningful units.
- Ensure embedding pipeline generates vector representations without error using the BAAI/bge-base-en-v1.5 model.
- Check if folder watcher detects new documents and updates the vector store accordingly.
- Execute basic user prompts to test retrieval + generation flow using context from the local knowledge base.

**Results:**

Core modules behaved as expected during isolated and integrated runs. Chunking preserved semantic boundaries. Folder monitoring dynamically updated the knowledge base. Prompt responses were accurate within the scope of loaded documents.

### 7.2.2 Performance Testing:

Objective: Measure efficiency of current implementation for small-to-medium scale academic material.

**Test Scenarios:**

- Evaluate vector generation and chunking times for 50-page PDFs.
- Observe response times for prompt queries post-embedding.

**Results:**

Chunking and embedding averaged ~0.3s per page. Prompt response times were under 2s for queries on documents up to 200 chunks, indicating acceptable performance for individual learners.

### 7.2.3 Security Testing (To Be Done):

While security features like user authentication and data isolation are planned, they have not yet been implemented. Future iterations will consider local data encryption and access control mechanisms to ensure privacy and compliance with academic data handling norms.

**Usability Testing:**

Objective: Assess ease of document ingestion and retrieval-based interaction.

**Test Scenarios:**

- Run end-to-end testing for document uploads followed by querying.
- Test prompt types: definitions, summaries, concept explanations.

**Results:**

Usability tests showed intuitive workflows. Users were able to place files and receive answers without manual intervention. AI responses were perceived as useful and contextually grounded in the loaded documents.

### 7.2.4 Key Observations:

- Modularity: The architecture supports scalable integration of future features like dashboards, LLM switching, and summarization views.
- Accuracy: Responses reflected document content accurately when queries matched chunk-level context.
- Extensibility: Design allows for easy augmentation of subjects, folders, or new retrieval logic.

The testing phase reflects a solid foundation for SmartScholar's core modules. While not all planned features are implemented, current components function reliably. Continued development and testing will further enhance the system's capability as an AI-powered academic study assistant.

# Chapter 8 CONCLUSION AND FUTURE WORK

## 8.1 Conclusion

The AI-Powered Study Assistant is a smart and user-friendly tool designed to help students learn better and faster. It automatically keeps track of all your study materials, such as notes, PDFs, and images, and updates itself whenever you add something new. When you ask a question, the assistant finds the most relevant information from your own materials and gives you clear, easy-to-understand answers.

Unlike regular AI tools like ChatGPT or Google, this assistant is personalized just for you. It learns from your materials, provides summaries, explains tough concepts, and even gives you emotional support and motivational tips. It can suggest helpful videos, articles, and other resources, and it helps you stay organized by giving feedback after each study session. You can also upload your notes for peer review and get writing help for your assignments.

Overall, this assistant makes studying easier, more organized, and more effective by combining the latest AI technology with features that support your learning, motivation, and well-being.

## 8.2 Future Work

To make the AI-Powered Study Assistant even better, here are some features we plan to add:

1. **Support for More Types of Content:**
   The assistant will be able to handle images, videos, audio files, and interactive diagrams, not just text.
2. **Automated Video Summaries:**
   After each session, the assistant can create a short video explaining what you learned, making it easier to review.
3. **Better Session Summaries and History:**
   It will automatically make notes and summaries after each session and save your

learning history in the cloud for easy access.

4. **Personalized Tests and Exercises:**
   The assistant will create quizzes and practice tests based on your progress and study materials, helping you prepare for exams.
5. **Group Study Support:**
   Multiple students can use the assistant together, making group study sessions more interactive and productive.
6. **Integration with Learning Platforms and Cloud Storage:**
   The assistant will work smoothly with platforms like Moodle and Google Classroom, and automatically sync your materials from Google Drive, OneDrive, etc.
7. **Progress Tracking and Smart Suggestions:**
   It will track your progress, suggest what to revise, and motivate you with quiz games and achievement badges.
8. **Privacy and Security:**
   Features will be added to protect your data and make sure everything is safe and private.

These improvements will make the AI-Powered Study Assistant a complete and reliable companion for all your learning needs, helping you and your friends study smarter and achieve better results.

# BIBLIOGRAPHY


- T. B. Brown et al., "Language Models are Few-Shot Learners," in Advances in Neural Information Processing Systems 33 (NeurIPS 2020), 2020.
- Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-Augmented Generation for Large Language Models: A Survey," arXiv, 2024. doi: arXiv:2312.10997.
- Y. Ding, W. Fan, L. Ning, S. Wang, H. Li, D. Yin, T.-S. Chua, and Q. Li, "A Survey on RAG Meets LLMs: Towards Retrieval-Augmented Large Language Models," arXiv, 2024. doi: arXiv:2405.06211.
- X. Lin, S. Yang, W. Wang, F. Feng, and T.-S. Chua, "Data-efficient Fine-tuning for LLM-based Recommendation," arXiv, 2024. doi: arXiv:2401.17197.
- B. Liu et al., "MFTCODER: BOOSTING CODE LLMS WITH MULTITASK FINE-TUNING," arXiv, 2023. doi: arXiv:2311.02303.
- L. Liu, H. Li, Y. Su, D. Cai, and Y. Wang, "A Survey on Retrieval-Augmented Text Generation," arXiv, 2022. doi: arXiv:2202.01110.
- E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models," arXiv, 2021. doi: arXiv:2106.09685.
- T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs via Low-Rank Adapters," arXiv, 2023. doi: arXiv:2305.14314.